AppWeaver: Agentic Discovery and Composition of O-RAN Apps for AI-Native 6G

Yongchao Dang, Tao Chen, and Riku Jäntti

Abstract—Open RAN (O-RAN) opens the radio access network to a growing ecosystem of third-party rApps and xApps, allowing radio resource optimization through standardized Non-Real-Time (Non-RT) and Near-Real-Time (Near-RT) RAN Intelligent Controllers (RICs). However, building O-RAN services for emerging 6G verticals, such as connected vehicles and immersive VR/XR, remains difficult because operators must still manually discover suitable applications, understand their dependencies, configure and validate them, and coordinate them into end-to-end workflows. This article introduces AppWeaver, an agentic-AI framework that orchestrates end-to-end O-RAN workflows by using LLM-based agents to interpret operator intent, query an Agent Card registry that describes each xApp and rApp in terms of its functions, properties, and interfaces, reason about their capabilities and dependencies, and compose them into ordered, service-specific workflows. In addition, AppWeaver confines LLM agents to workflow-level orchestration over existing xApps and rApps, rather than direct RAN control, preserving a clear control-plane boundary in which native RIC applications execute low-level RAN control and monitoring actions, while agent-generated workflows are checked against the applications' declared dependencies, interfaces and control scopes and dry-run on an E2 twin before deployment. The workflows are orchestrated with LangGraph, Agent Cards are served over Agent-to-Agent (A2A) and application tools over the Model Context Protocol (MCP), and AppWeaver is evaluated on a FlexRIC testbed, a 5G-LENA ns-3 RAN and a real transport link. Over 8,712 runs on three intent sets, AppWeaver composes the correct workflow for 69–94% of intents and 55–94% of those no operator template solves, where rule-based and catalog-prompted LLM baselines score 0%; deployed, its workflows measurably change throughput, energy and delay.

Index Terms—O-RAN, agentic AI, large language models (LLMs), workflow composition.

I. Introduction

Upcoming 6G networks will evolve into programmable, artificial intelligence (AI)-native platforms for immersive, sensing, and edge-intelligence services, whose diverse requirements demand flexible and service-aware RAN automation [1]. To provide this flexibility, the O-RAN ALLIANCE[1] defines a disaggregated and programmable RAN architecture with standardized open interfaces and hierarchical RAN Intelligent Controllers (RICs), where rApps in the Non-Real-Time (Non-RT) RIC support long-term optimization and policy guidance, while xApps in the Near-Real-Time (Near-RT) RIC provide near-real-time RAN control [2]. This openness reduces vendor lock-in and enables an open ecosystem of interoperable third-party xApps and rApps for service-specific RAN automation, providing end users with more adaptive, service-aware connectivity.

Building on this openness, Hexa-X-II[2] advances intent-based management for 6G operations, where operator intents are realized through the available xApps and rApps. However, as the application catalog grows, realizing intents across a multi-vendor ecosystem raises three open problems. First, an intent must be mapped to the right xApps, rApps, APIs, and control procedures, but O-RAN defines application exposure rather than intent-driven selection and composition[3], leaving this step manual. Second, an intent may span multiple applications that depend on shared data, models, policies, and control actions, yet these dependencies are still validated in separate steps across design, deployment, and runtime. Third, an intent can trigger automated control, but conflict mitigation, authorization, lifecycle control, auditing, rollback, and human approval are not tied together into one governed execution path. These gaps make intent realization difficult to scale and govern in open, multi-vendor O-RAN systems.

To this end, this paper proposes AppWeaver, an agentic-AI framework that orchestrates end-to-end O-RAN workflows by using LLM-based agents to interpret operator intent, discover reusable xApps and rApps, reason about their capabilities and dependencies, and compose them into ordered, service-specific workflows. The main design elements and contributions of AppWeaver are summarized as follows.

- Agent Card for xApp/rApp. Each xApp and rApp is wrapped by a machine-readable Agent Card[4] that describes its functions, interfaces, data dependencies, control scope, timescale, conflicts, lifecycle state, and tools. This allows agents to discover, compare, and reuse third-party applications without modifying their internals, while providing the semantic ground-

Yongchao Dang and Riku Jäntti are with the Department of Information and Communications Engineering, School of Electrical Engineering, Aalto University, 02150 Espoo, Finland (e-mail: firstname.lastname@aalto.fi).

Tao Chen is with VTT Technical Research Centre of Finland, 02150 Espoo, Finland (e-mail: tao.chen@vtt.fi).

[1]https://www.o-ran.org/

[2]https://hexa-x-ii.eu/

[3]https://www.o-ran.org/specifications

[4]https://www.agentcard.net/

TABLE I: Classification of representative agentic O-RAN systems by capability.

System	Control paradigm	Intent	Reuse & composition		Conflict & assurance			Evaluation
			Disc.	Comp.	Confl.	Valid.	Adapt.	
RIC-O [2]	Optimization-based RIC placement	✗	✗	✗	✗	✗	∼	Kubernetes
AutoRAN [3]	LLM-driven deployment automation	✓	✗	∼	✗	✓	∼	OAI/srsRAN/ARC
OREO [4]	Optimization-based xApp deployment	✗	∼	∼	∼	✗	✗	Colosseum emulator
RAN assurance [5]	Multi-agent assurance	∼	✗	∼	✓	✓	✓	Simulation
AGORAN [6]	Multi-agent intent negotiation	✓	∼	✗	✓	✗	∼	OAI/FlexRIC
AgentRAN [7]	Hierarchical multi-agent control	✓	✗	∼	∼	∼	✓	OAI (OTA)
Multi-scale [8]	Multi-timescale multi-agent control	✓	✗	✗	∼	✗	✓	Architecture
Multi-agentic rApp [9]	Multi-agent policy control	✓	✗	∼	✓	∼	∼	Simulation
MX-AI [10]	Multi-agent observability and control	✓	✗	∼	✗	∼	∼	OAI/FlexRIC
AppWeaver (ours)	Multi-agent workflow orchestration	✓	✓	✓	✓	✓	∼	FlexRIC/ns-3

Control paradigm: control approach adopted. Intent: natural-language interface. Disc.: discovery of reusable xApps/rApps. Comp.: composition into executable workflows. Confl.: cross-application conflict handling. Valid.: pre-deployment sandbox or twin validation. Adapt.: runtime adaptation and rollback. Marks: ✓ implemented and evaluated in the cited work; ∼ described or implemented in part, without evaluation; ✗ not addressed.

ing needed for dependency reasoning and compatibility checking.

- Agentic workflow composition. Based on the Agent Card abstraction and the Agentic Resource Discovery (ARD) principle[5], LLM agents translate operator intent into workflows by discovering suitable xApps and rApps, reasoning about their capabilities and dependencies, and coordinating execution through a LangGraph-based state machine. This enables reusable O-RAN applications to be selected, connected, and orchestrated for different service requirements without manual workflow construction.
- Agentic workflow testing and deployment. Each generated workflow is tested before deployment by compatibility checks against the Agent Cards, a lease check, and a dry-run of its E2 control steps on a twin RIC; only workflows that pass are deployed, every step is logged, and a network twin is evaluated as a KPI gate.

These agentic capabilities enable an intent-to-deployment pipeline that covers application discovery, workflow composition, predeployment validation, and governed execution across reusable O-RAN applications. The remainder of this paper is organized as follows. Section II reviews related work on agentic AI for O-RAN. Section III introduces the AppWeaver architecture. Section IV presents the prototype and its evaluation on FlexRIC, ns-3 and a real transport link. Section V concludes the paper and outlines future directions.

II. Agentic AI for O-RAN

Agentic AI for O-RAN refers to governed LLM-based agents that interpret operator intent, retrieve external knowledge, reason over network context, plan actions, and invoke O-RAN APIs and tools for automation. Unlike traditional rule-based network agents that follow predefined policies and fixed control logic, these agents interpret natural-language instructions and adapt their behavior to dynamic network conditions. Recent work explores this direction from three complementary perspectives, namely xApp and rApp development, O-RAN component deployment and lifecycle management, and intent-driven network automation, summarized in Table I.

A. Agentic AI for xApp and rApp Development

Developing xApps and rApps requires a heavy engineering burden, as each application requires RAN control logic, E2 and A1 service model integration, RIC SDK interfacing, and validation on the target platform. Agentic AI is being leveraged to reduce this burden, with LLM-based agents translating natural-language requirements into application code, service model bindings, configuration artifacts, and test templates, and supporting adaptive reasoning inside xApps for radio resource management [11]. However, these efforts mainly target the development of individual applications and treat the LLM as a code-generation or reasoning engine, while agentic capabilities such as tool invocation, knowledge retrieval, multi-step planning, and multi-agent collaboration remain underexplored.

B. Agentic AI for O-RAN Deployment and Management

O-RAN deployment and management involve placing disaggregated functions across cloud-native infrastructure, allocating resources, resolving inter-application conflicts, and verifying behavior before execution. To automate these tasks, existing studies combine optimization, learning-based prediction, and agentic decision-making. RIC-O optimizes the placement of distributed near-RT RIC instances across the cloud-edge continuum through dynamic clustering of radio nodes [2], while graph neural network based methods predict conflicts among xApps

[5]https://agenticresourcediscovery.org/

before deployment [12]. AutoRAN translates high-level operator intent into deployment, configuration, and testing artifacts for multi-vendor O-RAN systems [3], whereas OREO formulates xApp deployment as a joint optimization problem over application selection, function sharing, configuration, and resource allocation [4]. Assurance-oriented frameworks add explicit verification stages to validate AI-generated decisions before execution [5]. Although these efforts reduce the operational complexity of deploying and managing O-RAN components, they mainly assume that the candidate applications are already known and focus on their placement, configuration, or validation, rather than on discovering reusable xApps and rApps and assembling them into service-specific workflows.

C. Agentic AI for Intent-Based RAN Automation

Intent-based RAN automation enables operators to specify high-level service objectives in natural language, which collaborating agents translate into control actions; a key challenge is how such intents are interpreted and resolved in multi-stakeholder settings. AGORAN addresses this challenge through intent negotiation, where LLM-based broker agents reconcile conflicting requirements under governance constraints and translate the agreed outcomes into enforceable policies through standard O-RAN interfaces [6]. Building on this direction, subsequent work aligns agent structures and computational models with O-RAN's hierarchical, multi-timescale control loops, as in AgentRAN, which coordinates rApp, xApp, and dApp agents through A2A communication and MCP-based tool invocation [7], and multi-scale frameworks that map LLMs, small language models (SLMs), and wireless foundation models to the Non-RT, Near-RT, and real-time loops to meet heterogeneous latency requirements [8]. The direction further extends to concrete implementations and system-level demonstrations, including conflict-aware policy generation through collaborative rApp agents [9], [13] and live prototypes such as MX-AI, which provides end-to-end observability and closed-loop control in operational deployments [10]. Like AgentRAN, AppWeaver exposes its applications over A2A and MCP, but its agents do not control the RAN themselves: they select and chain existing xApps and rApps, using Agent Cards that declare O-RAN-specific service models, data dependencies, control scope and timescale, check every composed chain against these declarations before deployment, and dry-run it on a twin.

Despite advances in application development, deployment automation, intent understanding, architectural design, and closed-loop execution, existing systems commonly assume that the applications required to realize a service are already known. However, an end-to-end service often requires coordinating multiple reusable applications rather than executing a single xApp or rApp in isolation. For example, traffic steering typically involves KPI monitoring, anomaly detection, QoE prediction, traffic-steering optimization, and RAN control [14]. Current approaches therefore map operator intent to actions over predefined xApps and rApps, but do not address how such applications are discovered in an open, multi-vendor ecosystem, verified for interface and control-scope compatibility, and composed into validated executable workflows. As O-RAN application ecosystems expand and services increasingly span multiple vendors, this gap in application discovery and composition becomes a key barrier to fully autonomous service orchestration, motivating the design of AppWeaver.

III. AppWeaver Architecture and Workflow

A. AppWeaver Architecture

Figure 1 shows the AppWeaver architecture, which sits between the operator specifying the intent and the RAN vendors providing the RIC and its xApps and rApps. AppWeaver does not replace existing O-RAN applications but orchestrates them into service-specific workflows through discovery, composition, validation, and governed execution. At its core, a LangGraph-based orchestration layer[6] interprets the operator intent and coordinates four orchestration agents. The Discover agent identifies candidate xApps and rApps, the Compose agent has the LLM planner assemble them into an ordered workflow, the Validate agent checks the workflow before deployment, and the Deploy agent applies the validated workflow to the RIC. These agents draw on an Agent Registry that stores the Agent Card of each available xApp and rApp, also served over A2A[7], and on a shared state that holds the workflow context. Discovery is grounded in the registry by BM25 retrieval over the Agent Cards. Fig. 1 is the reference design; the prototype passes LangGraph state between the agents instead of an A2A bus, and its memory, SLM and feedback loop remain future work.

AppWeaver operates across two environments, the live RAN and a network digital twin. On the live RAN, a per-application domain agent wraps each selected xApp or rApp and launches its monitoring and control tools, which an MCP server[8] also exposes to external agents, so the domain agent orchestrates the workflow while the application executes the E2 and A1 control actions. Before deployment, the Validate agent uses a digital twin at two levels: an E2 twin (a second RIC with an emulated E2 node) on which every E2 control step is dry-run to catch interface, lease and acknowledgement errors, and a network twin (a RAN simulator) that predicts the workflow's KPI impact; the prototype gates on the E2 twin, and the network twin is evaluated as a KPI gate in Section IV. Only workflows that pass are applied to production. This separation gives AppWeaver a safety boundary by testing workflows before they reach the live RAN, while keeping all E2 and A1 control actions within existing xApps and rApps and their native control loops.

[6]https://www.langchain.com/langgraph
[7]https://a2a-protocol.org/latest/specification/
[8]https://modelcontextprotocol.io/specification/

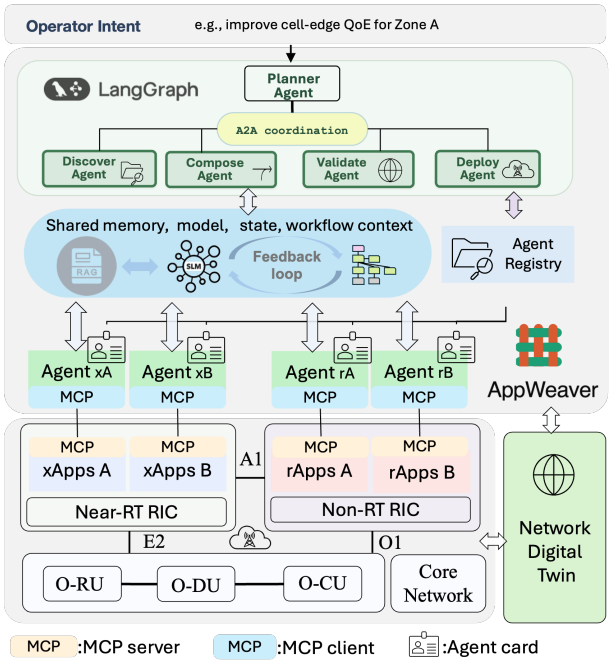


Fig. 1: AppWeaver reference architecture (the prototype's scope is given in Section III-A).

B. AppWeaver Workflow

The AppWeaver workflow moves through four stages, discovery, composition, validation, and deployment, all driven by the LLM-based agents in the AppWeaver core; Fig. 2 shows them for a QoE-steering intent, taken from a recorded run. Given an operator intent, the Discover agent retrieves candidate applications from the Agent Registry by BM25 search over the Agent Cards, matching the intent to advertised capabilities rather than names; the intent in Fig. 2 names no application, and the 19 retrieved cards include a QoE-prediction rApp, a traffic-steering rApp, a UE QoS-control rApp and their deprecated releases.

intent	"Steer at-risk video users away from the congested cell before their QoE drops." (no skill or app named)
discover	BM25 over 80 executable Agent Cards, k=8 rApp families → 19 candidates in 5 ms, incl. qoe_prediction.rApp, traffic_steering.rApp, ue_qos_control.rApp and the deprecated v0 variants.
compose	Planner (gpt-oss-120b, 31.0 s): KPM → qoe_prediction → traffic_steering → RC; v0 variants rejected.
check	Card checks: dependency, interface, control scope, timescale, lifecycle → 0 errors, 2 warnings (second RC writer serialised; near-RT step after non-RT step).
validate	Twin dry-run of the RC step over E2 on the twin RIC: acknowledged in 7.2 s; no busy lease → deployable at 38.2 s.
deploy	Four calls in chain order (KPM observe, QoE predict, steering policy, RC control): 4/4 acknowledged, lease wait 0 s.
baselines	Rule router and catalog LLM: KPM → SLICE → TC → RC (wrong, 3/3 repeats); AppWeaver correct 3/3.

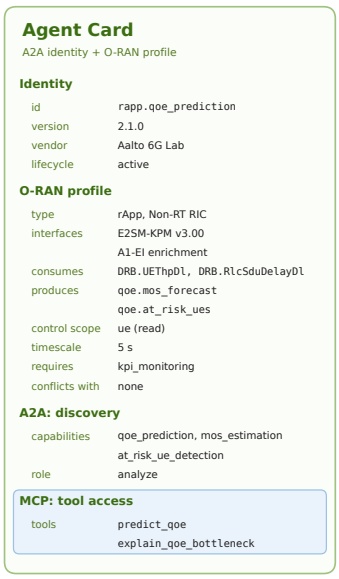
Fig. 2: AppWeaver trace for a QoE-steering intent, from a recorded run (Batch B, 81-card registry).

The Compose agent has the LLM planner turn these candidates, their actions, and the operator constraints into an ordered workflow: here KPM monitoring, then QoE prediction, then traffic steering, then RAN control, rejecting the deprecated releases. The chain is then checked against the Agent Cards: unmet dependencies, writers declared in conflict and deprecated releases are errors, repaired from the cards (inserting a missing producer, dropping a conflicting writer, replacing a deprecated release) or else rejected; interface and timescale mismatches and a second writer on one resource, which the scheduler serializes, are warnings. The Validate agent then dry-runs each E2 control step on a twin RIC and checks that no other workflow holds the applications' leases, and the Deploy agent launches the steps in order, writing each result, including the application's E2 control acknowledgement, back into the shared state; if an application is busy, a bounded Recover step waits or re-plans once. KPI prediction on a network twin is evaluated in Section IV.

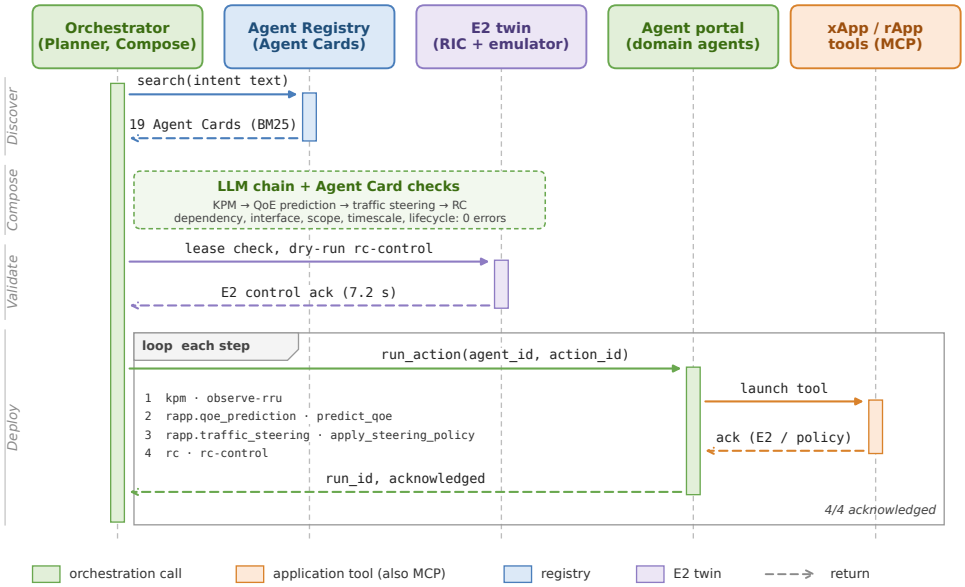
C. Agent Interfaces and Procedure

Figure 3 details the AppWeaver interfaces for exposing, discovering, and invoking third-party xApps and rApps, using the QoE-steering intent of Fig. 2 as an example. Fig. 3(a) shows the Agent Card, a machine-readable JSON descriptor registered in the Agent Registry that lets AppWeaver treat each xApp or rApp as a discoverable and reusable component. Besides its A2A identity and capabilities, it carries an O-RAN profile: application type and RIC, service models and A1 interfaces (e.g., E2SM-KPM, A1 enrichment information), the data it consumes and produces, its control scope (cell, UE, slice, node or transport; read or write), its timescale and RIC tier, the capabilities it requires upstream, the applications it conflicts with, and its lifecycle state; the MCP section lists the tools through which the application is monitored and controlled. These fields drive the compatibility checks of Section III-B. Because this description is maintained outside the application, AppWeaver translates operator intent into RAN control through existing xApps and rApps without modifying their internal logic or driving O-RAN interfaces directly. A new application is onboarded by registering its Agent Card.

Fig. 3(b) follows the QoE-steering intent of Fig. 2 through the four stages. The orchestrator searches the Agent Registry with the intent text alone, without a skill hint; BM25 retrieval returns 19 Agent Cards, among them the QoE-prediction and traffic-steering rApps. The Compose agent orders KPM monitoring, QoE prediction, traffic steering and RAN control, and the card checks report no errors. The Validate agent checks that no other workflow holds a lease on the selected applications and dry-runs the RAN-control step on the E2 twin, which acknowledges it after 7.2 s. The Deploy agent then calls each step's domain agent with the step's agent and action identifiers; the domain agent launches the corresponding



(a) Agent Card of a registry entry



(b) Discovery, validation and invocation sequence of the run in Fig. 2

Fig. 3: AppWeaver agent interfaces and procedure.

application tool and reports the run and its acknowledgement, read from the application log (the E2 control acknowledgement for the RC xApp, the written policy for an rApp). All four steps were acknowledged. Each tool is the same function that the application's MCP server exposes to MCP clients (tools/list, tools/call), so an external agent can reach it through MCP; in the measured runs, the domain agent launches it directly. Agent Cards are served over A2A (agent.get_card). This keeps the orchestrator at the workflow level: it never drives E2 or A1 itself, and every external operation, whether observation, prediction or control, is an explicit typed action that is logged, validated and replayable.

The Agent Card and MCP abstractions expose heterogeneous xApps and rApps as discoverable, reusable agents, and one planning call, card-based repair and a bounded recover step compose them into a validated workflow. AppWeaver serves as an intent-translation layer between the operator and vendor-provided RIC applications, mapping high-level objectives onto existing xApps and rApps through their agent interfaces. This layering separates timescales, with AppWeaver orchestrating at seconds to minutes while the applications retain native E2 and A1 control in their own loops, keeping orchestration separate from control execution.

IV. AppWeaver Evaluation

A. Prototype Setup

The AppWeaver LangGraph orchestrator runs next to a FlexRIC Near-RT RIC on one host; the Planner uses gpt-oss-120b[9] served by vLLM on an NVIDIA DGX Spark (GB10, 128 GB unified memory), with reasoning effort medium, temperature 0.2 and a fixed seed. The real RIC applications are four FlexRIC xApps used over E2, KPM monitoring (E2SM-KPM), slice control (SLICE), transport control (TC) and RAN control (E2SM-RC), plus MAC, RLC, PDCP and GTP monitors; together they form the 9-card paper registry. The rApps (QoE prediction, traffic steering, energy saving and 29 other families, each with vendor variants, deprecated releases and distractors) are emulated decision services behind standard tool interfaces; with them the registry grows to 81 Agent Cards. The E2 node used for the composition benchmark is FlexRIC's emulator, whose KPM values are synthetic; the Validate agent dry-runs each E2 step on a second, independent RIC and emulator (the E2 twin), which checks interfaces, leases and acknowledgements. Network effect is therefore measured separately, on two substrates with real dynamics: a 5G-LENA ns-3 RAN (four gNBs at 3.5 GHz) attached to FlexRIC over E2SM-KPM/RC, where a second ns-3 instance with the same layout but a different seed acts as the KPI twin and rehearses the RC commands decided on the live E2 KPM, and a real Linux transport bottleneck for TC, driven by an E2 agent whose TC service model applies the received queue control to the link. All code, intents, Agent Cards and run records are public[10].

[9]https://huggingface.co/openai/gpt-oss-120b
[10]https://github.com/Dangy1/flexric-py310/tree/appweaver-revision

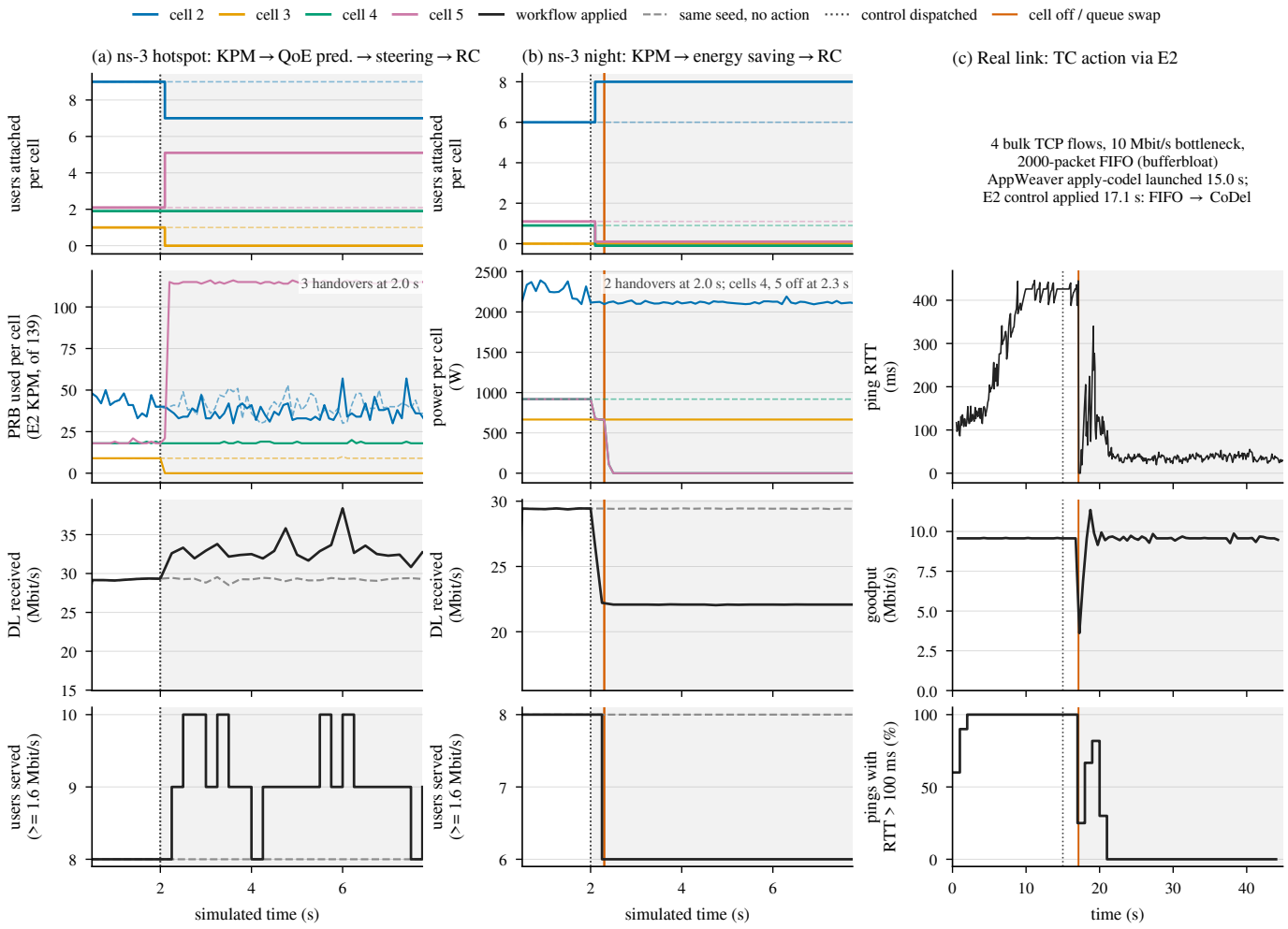


Fig. 4: System KPIs while a composed workflow acts (live instance; dashed: same seed without the action; shaded: after the action). Rows: users attached per cell and cell load or power from the E2 KPM stream, received downlink throughput, and users served (at least half their offered rate). (a) ns-3 hotspot, KPM→QoE prediction→traffic steering→RC, seed 104; (b) ns-3 night load, KPM→energy saving→RC; (c) real 10 Mbit/s link, AppWeaver's TC action (FIFO→CoDel) sent over E2: RTT, goodput and share of pings above 100 ms.

B. Benchmark and Methods

Three intent sets are used. Batch A has 50 operator intents over the paper registry: 11 match one of the three operator workflow templates (known) and 39 do not (open-world), 27 of which no template solves even partially. Batch B has 109 intents over the 81-card registry and adds 59 unseen-capability intents that need an rApp absent from every template; Discover adds the top 8 rApp families by BM25 to the xApps, and an ablation puts all 81 cards in the prompt. Batch C has 60 held-out intents written after the design was frozen (30 known, 30 open-world). Each intent is labeled with the applications it requires and their partial order; a composed workflow is correct if it contains every required application in a valid order and any extra step is read-only (rule accuracy, defined before the runs). An intent can therefore have several valid workflows. Six intents in A and B carry a seeded conflict: before the method runs, another operator's workflow starts a write action on the TC xApp (five intents) or the SLICE xApp (one) and holds its lease for 30 s; correct handling means issuing no control to the busy xApp, then waiting or re-planning, never preempting it. AppWeaver is compared with five methods, all using the same planner model: a rule router, which maps intent keywords (latency, slice, transport, monitoring) to one of the three operator templates; a single-app router, which runs the one application whose keywords match best; an LLM without the catalog, which names applications from its own knowledge; an LLM with the full catalog in its prompt, which returns a chain in one pass that is executed as returned, without Agent Card checks; and AppWeaver without repair. A tool call counts as successful only if the target application acknowledges it (E2 control acknowledgement, KPM indication or rApp

reply); launching a process is not enough. Each intent runs 5 (A) or 3 (B, C) times, 8,712 records in total, with planner caches off.

C. Workflow Composition

Fig. 5(a) shows that AppWeaver composes the correct workflow for 70.4% of the Batch A intents, 68.8% in Batch B and 93.9% in the held-out Batch C, against 34.8%, 16.8% and 53.3% for the catalog-aware LLM and 36.0%, 16.5% and 20.0% for the rule router. The baselines score well only where a template fits: on the intents no template solves, both score 0% in every batch, while AppWeaver reaches 67.4%, 55.6% and 93.9% (Fig. 5(b)); on Batch B's unseen-capability intents it reaches 82.5%. Fig. 2 shows one such case: both baselines return the generic KPM→SLICE→TC→RC template, while AppWeaver composes the QoE-prediction and steering rApps. Retrieving 8 of 81 cards gives the same accuracy as showing all of them (68.8% both) with about one third of the prompt tokens (9k versus 25k) and a median planning time of 32.5 s instead of 36.2 s. Adding the operator templates and an experience memory of earlier plans as retrieved examples raises accuracy to 74.6% (B) and 96.1% (C). A second planner, GLM-5.3-Flash, gives the same accuracy on Batch C (93.9%) and the same 0% for the template-free baselines, so the gain is not specific to one model.

D. Network Effect of the Composed Workflows

Fig. 4 shows the KPIs while a workflow acts, Fig. 5(c) the change it causes; the rApps of the composed chain decide on the live KPM snapshot, the twin rehearses the resulting UE-level RC commands, and live executes them. On the ns-3 hotspot, QoE steering hands users from the loaded cell to a neighbor at $t = 2$ s, whose load rises within 0.3 s; over 15 distinct decisions the source cell's PRB falls by 33%. Throughput, depending on which users move, changes by -15 to $+11$% ($+2.1$% on average), users served by $+5.0$%. At night, energy saving switches two cells off in the four seeds where the snapshot meets its rule: energy falls by 41%, but two users lose service, cutting throughput and users served by 25%. On the real link, the TC action reaches the queue 2.1 s after launch; in the four repeats where four TCP flows build a queue it cuts the median RTT from about 420 ms to 35 ms (pings above 100 ms: 100%→4–7%), in the fifth from 64 to 33 ms, at unchanged goodput.

Rehearsing the live decision, the twin predicts steering within 3–7 percentage points on average (19 distinct decisions) and energy saving within 1 point. Deciding on its own state, it moves other users and errs by 10–30 points. Identical re-runs differ by under 1 point; the rest is mobility and fading the twin cannot know. The twin's accept/reject verdict under the pre-declared rule matches the live outcome in 14 of 19 decisions (74%, 95% CI 51–88%), no better than with its own decision (8/10): the remaining error is as large as the rule's 5% margin. Four of the five misses are conservative. It stops the harmful energy policy only if its rule tests users served or throughput. A prediction takes about 5 min, enough for Non-RT policies only.

E. Reliability, Conflicts and Latency

Across all batches, 97–100% of AppWeaver's tool calls (572, 836 and 380 in A, B and C) and 96–100% of its control calls are acknowledged by the target application, and no plan, even before repair, references an application outside the registry, whereas the LLM without the catalog produces 705 such references in Batch A. On seeded conflicts AppWeaver never preempts the busy application: it waits for the lease and completes in 37% (A) and 44% (B) of seeded runs (one run with all 81 cards stays unresolved), while the rule router runs into the lease and completes with the wrong workflow in 83% of its cases. The time from intent to deployable workflow, measured on every record (250, 327 and 180 per method in A, B and C, planner caches off), has a median of 26–36 s and a 95th percentile of 55–64 s; the planner dominates and the E2 dry-run adds about 8 s. This is slower than the single-pass LLM (7–9 s) but within the Non-RT RIC timescale (>1 s)[11].

V. Conclusion and Future Work

This article presented AppWeaver, an agentic-AI framework that translates operator intent into validated O-RAN workflows by discovering reusable xApps and rApps, composing them, and deploying them after validation. AppWeaver wraps each application in a machine-readable Agent Card for capability-based discovery, coordinates Discover, Compose, Validate, and Deploy agents in a LangGraph orchestrator, exposes applications over A2A and MCP, and dry-runs every workflow's E2 control steps on a twin before it reaches the live RAN. By confining LLM agents to workflow-level orchestration, AppWeaver keeps all E2 and A1 control actions inside the native xApps, rApps, and RIC control loops, so reasoning and coordination sit above the RAN while execution remains within standardized interfaces, and every step is logged. The evaluation supports this design. Over three intent sets and registries of up to 81 Agent Cards, AppWeaver composes the correct workflow for 69–94% of intents and for 55–94% of those no operator template solves, where the baselines score 0%, with grounded, acknowledged control calls and no pre-emption of busy applications, at a latency within the Non-RT timescale. Deployed on a 5G RAN model and a real transport link, the composed workflows change throughput, energy and delay measurably, and the network twin predicts these effects closely enough to stop a harmful policy, provided its acceptance rule tests the KPIs that policy hurts.

[11] O-RAN.WG1.OAD, https://specifications.o-ran.org/.

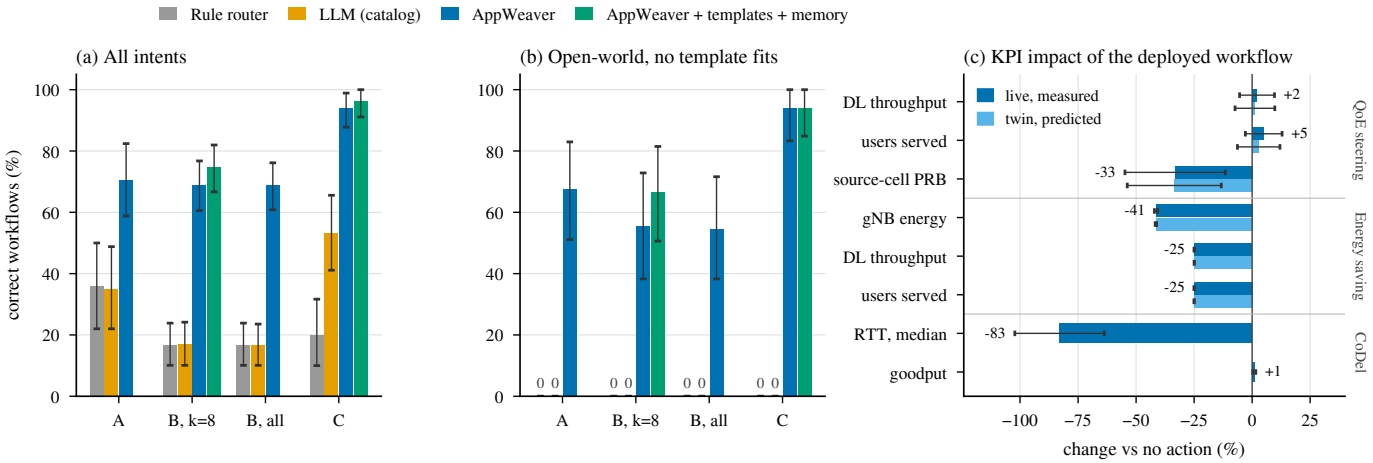


Fig. 5: (a) Rule accuracy per batch and (b) on open-world intents that no operator template solves, 95% intent-clustered bootstrap CIs; (c) KPI change caused by the deployed workflow versus the same instance without it, live measurement and twin prediction, mean ± sd over distinct decisions (15 steering, 4 energy saving) and 5 real-link repeats (no twin).

Several directions remain for future work. First, runtime adaptation is currently partial, and tighter closed-loop self-healing together with learned digital-twin surrogates would strengthen the observe-and-adapt stage and reduce validation cost. Second, discovery is performed over a curated Agent Card catalog, and scaling to large multi-vendor repositories and open marketplaces raises trust, versioning, and conflict-resolution challenges. Third, rApps are emulated as test endpoints in the current testbed, and deploying AppWeaver against production-scale Non-RT RIC applications in collaboration with industry partners is a natural next step. Finally, the A2A and MCP bindings used here are pragmatic mechanisms, and aligning them with the RAN-grade requirements on semantic contracts, authorization, behavioral bounds, traceability, and rollback now being studied in O-RAN working groups and 3GPP SA5 is important work toward a standardized agent-to-O-RAN interaction model.

References

[1] M. Polese, N. Mohamadi, S. D'Oro, L. Bonati, and T. Melodia, "Beyond connectivity: An open architecture for AI-RAN convergence in 6G," IEEE Communications Magazine, vol. 64, no. 7, pp. 90–95, Jul. 2026.

[2] G. M. Almeida, G. Z. Bruno, A. Huff, M. Hiltunen, E. P. Duarte, C. B. Both, and K. V. Cardoso, "RIC-O: Efficient placement of a disaggregated and distributed RAN intelligent controller with dynamic clustering of radio nodes," IEEE Journal on Selected Areas in Communications, vol. 42, no. 2, pp. 446–459, Feb. 2024.

[3] S. Maxenti, R. Shirkhani, M. Elkael, L. Bonati, S. D'Oro, T. Melodia, and M. Polese, "AutoRAN: Automated and zero-touch open RAN systems," IEEE Transactions on Mobile Computing, vol. 25, no. 7, pp. 10 112–10 129, Jul. 2026.

[4] F. Mungari, C. Puligheddu, A. Garcia-Saavedra, and C. F. Chiasserini, "O-RAN intelligence orchestration framework for quality-driven xApp deployment and sharing," IEEE Transactions on Mobile Computing, vol. 24, no. 6, pp. 4811–4828, Jun. 2025.

[5] S. Singh, A. Bhat, Shweta M, S. K. Singh, M. Hong, Madhan Raj K, K. Sithamparanathan, S. A. Khowaja, and K. Dev, "Agentic AI for ultra-modern networks: Multi-agent framework for RAN autonomy and assurance," in Proc. IEEE Int. Conf. Commun. (ICC), Glasgow, United Kingdom, May 2026, pp. 1–6.

[6] I. Chatzistefanidis, N. Nikaein, A. Leone, A. Maatouk, L. Tassiulas, R. Morabito, I. Pitsiorlas, and M. Kountouris, "AGORAN: An agentic open marketplace for 6G RAN automation," Computer Networks, vol. 275, p. 111927, Feb. 2026.

[7] M. Elkael, S. D'Oro, L. Bonati, M. Polese, Y. Lee, K. Furueda, and T. Melodia, "AgentRAN: An agentic AI architecture for autonomous control of open 6G networks," IEEE Communications Magazine, 2026, early access.

[8] H. Navidan, M. Cheraghinia, J. Fontaine, M. Seif, E. De Poorter, H. V. Poor, I. Moerman, and A. Shahid, "Toward autonomous O-RAN: A multi-scale agentic AI framework for real-time network control and management," IEEE Network, 2026, early access.

[9] H. Li, Y. Wu, and D. Simeonidou, "Multi-agentic AI for conflict-aware rApp policy orchestration in open RAN," in Proc. IEEE Int. Conf. Commun. (ICC). Glasgow, United Kingdom: IEEE, May 2026, pp. 1–6.

[10] I. Chatzistefanidis, A. Leone, A. Yaghoubian, M. Irazabal, N. Sehad, L. Bariah, M. Debbah, and N. Nikaein, "MX-AI: Agentic observability and control platform for open and AI-RAN," in Proc. IEEE Int. Conf. Commun. (ICC), Glasgow, United Kingdom, May 2026, pp. 1–6.

[11] A. Maatouk, N. Piovesan, F. Ayed, A. De Domenico, and M. Debbah, "Large language models for telecom: Forthcoming impact on the industry," IEEE Communications Magazine, vol. 63, no. 1, pp. 62–68, Jan. 2025.

[12] M. Al Shami, J. Yan, and E. T. Fapi, "O-RAN xApps conflict prediction using graph convolutional networks," IEEE Transactions on Mobile Computing, vol. 25, no. 7, pp. 10 876–10 892, Jul. 2026.

[13] S. Chatzimiltis, M. B. Mashhadi, M. Shojafar, M. Debbah, and R. Tafazolli, "Agentic AI for 6G: A new paradigm for autonomous RAN security compliance," IEEE Communications Standards Magazine, 2026, early access.

[14] H. Yang, C. Mcpeak, R. Nagampally, N. Tripathi, G. Anderson, J. H. Reed, Y. Yang, and D. J. Jakubisin, "Agile 5G networks: Advance traffic steering xApp for interference mitigation," in Proc. IEEE Military Commun. Conf. (MILCOM). Washington, DC, USA: IEEE, Oct. 2024, pp. 300–305.